

03_consensus_signals

June 29, 2026

1 03 — Consensus signals

Step 3 of 3. Load the vetted roster, pull everyone's **open positions**, group by (market, outcome), compute the **independence-weighted consensus score** (Blueprint §4), apply the **price guard** (§5) and signal gate, and emit a ranked signal table with **suggested balanced sizing** (§8).

Run this **daily**. Output is a proposal you review and execute manually.

```
[9]: import importlib, pmc, json
importlib.reload(pmc)
from pmc import (CFG, get_open_positions, get_market, portfolio_value,
                 position_size_eur)
import pandas as pd, numpy as np

roster = json.load(open("roster.json"))["wallets"]
skill_by_wallet = {r["wallet"]: r.get("skill", 0.5) for r in roster}
print(f"loaded {len(roster)} roster wallets")
```

loaded 27 roster wallets

1.1 1. Pull open positions for every roster wallet

```
[10]: rows = []
for r in roster:
    w = r["wallet"]
    pos = get_open_positions(w)
    pv = portfolio_value(pos) or 1.0
    for p in pos:
        val = float(p.get("currentValue") or 0)
        rows.append({
            "wallet": w,
            "skill": skill_by_wallet.get(w, 0.5),
            "conditionId": p.get("conditionId"),
            "title": p.get("title"),
            "outcome": p.get("outcome"),
            "avgPrice": float(p.get("avgPrice") or 0),
            "curPrice": float(p.get("curPrice") or 0),
            "value": val,
```

```

        "conviction": min(CFG.CONVICTION_CAP, val / pv),
    })
# build with explicit columns so it works even when rows is empty (no KeyError)
_cols = [
    ↪["wallet", "skill", "conditionId", "title", "outcome", "avgPrice", "curPrice", "value", "conviction"]
pos_df = pd.DataFrame(rows, columns=_cols).dropna(subset=["conditionId", ↪
    ↪"outcome"])
# drop positions in markets that are effectively already resolved (price pinned ↪
    ↪near 0 or 1)
b = CFG.RESOLVED_PRICE_BAND
_before = len(pos_df)
pos_df = pos_df[(pos_df["curPrice"] > b) & (pos_df["curPrice"] < 1 - b)]
print(f"dropped {_before - len(pos_df)} near-resolved positions")
if pos_df.empty:
    print("No open positions found. Check notebook 02 produced a non-empty ↪
    ↪roster.json,")
    print("and that notebook 01's leaderboard smoke test returned rows.")
else:
    print(f"{len(pos_df)} position rows across {pos_df['conditionId'].
    ↪nunique()} markets")

```

dropped 6378 near-resolved positions
125 position rows across 105 markets

1.2 2. Independence weight (simplified v0)

A full version clusters wallets by historical co-movement. v0 uses a practical proxy: if many roster wallets pile into the *same side at a near-identical price*, we damp each one's weight (likely one shared idea / copy-bots). We refine this in the backtest phase.

```

[11]: def independence_weights(group: pd.DataFrame) -> pd.Series:
    # cluster by rounded entry price; wallets in a big same-price cluster get ↪
    ↪damped
    buckets = (group["avgPrice"] * 20).round() # 5-cent buckets
    counts = buckets.map(buckets.value_counts())
    # weight = 1 for unique entries, decaying toward INDEPENDENCE_FLOOR as ↪
    ↪cluster grows
    w = 1.0 / counts
    w = CFG.INDEPENDENCE_FLOOR + (1 - CFG.INDEPENDENCE_FLOOR) * w
    return w.clip(CFG.INDEPENDENCE_FLOOR, 1.0)

```

1.3 3. Consensus score per (market, outcome)

```

[12]: signals = []
for (cid, outcome), g in pos_df.groupby(["conditionId", "outcome"]):
    g = g.copy()
    g["indep"] = independence_weights(g)

```

```

g["contrib"] = g["skill"] * g["conviction"] * g["indep"]
score = g["contrib"].sum()
backers = len(g)
# dissent: roster wallets holding the OTHER outcome of the same market
other = pos_df[(pos_df["conditionId"] == cid) & (pos_df["outcome"] !=
↳outcome)]
dissenters = other["wallet"].nunique()
signals.append({
    "conditionId": cid,
    "title": g["title"].iloc[0],
    "outcome": outcome,
    "backers": backers,
    "dissenters": dissenters,
    "score": round(score, 4),
    "median_entry": round(g["avgPrice"].median(), 3),
    "cur_price": round(g["curPrice"].median(), 3),
})
sig_df = pd.DataFrame(signals).sort_values("score", ascending=False)
print(f"{len(sig_df)} candidate (market, outcome) groups")
sig_df.head(15)

```

112 candidate (market, outcome) groups

```

[12]:                                     conditionId \
87  0xbb4d51e6364066d92eb6f9b8413dd7193de709667360...
4   0x0bbc63372aaf0867db23fc361b4e37f7d3125c7acef6...
37  0x4f3421fb2daf5cca7430ed8d8132463963081572d754...
31  0x375409bc5eeeff961e82b479caeccc20f33d15738e5b...
67  0x8d8e0630bd63491f2a77d1e0fb0f2750ec4ff118151b...
5   0x0c4cd2055d6ea89354ffddc55d6dbcef9355748112ea...
38  0x502a94e5c525766d5ee7f16c6568131ba1b2cbadb69c...
26  0x30d55d8124ee1e12dabe89201badc45669b81dff69e4...
59  0x75abb5b9b5ef0ed3f468be0b91559f2a413b750b4a7a...
71  0x9345d5142a67f5541264c96515496affee02580f1c57...
76  0xa090ae1a88ca96bcae7523a65a96425ed53616709be7...
75  0x9f61a4c01f2c8414b54fae93f7bad383593edea835cc...
35  0x46abfdead852f9cd12fbd348b4255cbe09485410626b...
32  0x376f0a5ce8279b4bf840018217487848634b686a9669...
56  0x717c526bb6a5946f75423352cbbd0d92ab87651710a7...

                                     title                outcome \
87  Will the Iranian regime fall before 2027?                No
4   Boston Red Sox vs. New York Yankees    Boston Red Sox
37  Will Portugal win the 2026 FIFA World Cup?                No
31  Will England win the 2026 FIFA World Cup?                Yes
67  St. Louis Cardinals vs. Cincinnati Reds    Cincinnati Reds
5   Will Argentina win the 2026 FIFA World Cup?                Yes

```

38	Will United Russia (ER) gain the most seats in...	Yes
26	Will Brazil win the 2026 FIFA World Cup?	Yes
59	St. Louis Cardinals vs. Cincinnati Reds: O/U 9.5	Under
71	Billionaire one-time wealth tax passes in Cali...	No
76	Nothing Ever Happens: 2026	No
75	Will a team from LPL (China) win MSI 2026?	Yes
35	Brazil vs. Japan: Brazil O/U 1.5	Over
32	Will 2026 be the second-hottest year on record?	Yes
56	Boston Red Sox vs. New York Yankees: O/U 8.5	Under

	backers	dissenters	score	median_entry	cur_price
87	3	0	0.1282	0.544	0.905
4	2	0	0.1014	0.470	0.510
37	1	0	0.0818	0.880	0.936
31	2	0	0.0726	0.092	0.102
67	1	0	0.0724	0.530	0.615
5	1	0	0.0718	0.086	0.198
38	1	0	0.0718	0.693	0.585
26	1	0	0.0652	0.048	0.072
59	1	0	0.0596	0.479	0.515
71	1	0	0.0527	0.611	0.655
76	1	0	0.0520	0.615	0.175
75	1	0	0.0520	0.266	0.290
35	1	0	0.0492	0.494	0.550
32	1	0	0.0425	0.291	0.660
56	1	0	0.0422	0.510	0.675

1.4 4. Apply the signal gate + price guard (Blueprint §4–§5)

```
[13]: def gate(r) -> bool:
    price_ok = (
        r["cur_price"] <= r["median_entry"] + CFG.SLIPPAGE_TOLERANCE and
        CFG.PRICE_FLOOR <= r["cur_price"] <= CFG.PRICE_CEILING
    )
    return (
        r["backers"] >= CFG.MIN_INDEPENDENT_BACKERS and
        r["dissenters"] <= CFG.MAX_DISSENTERS and
        r["score"] >= CFG.CONSENSUS_SCORE_CUTOFF and
        price_ok
    )

live = sig_df[sig_df.apply(gate, axis=1)].copy()
print(f"{len(live)} signals pass the full gate")
```

0 signals pass the full gate

1.5 5. Suggested balanced sizing

We estimate the smart-money-implied probability conservatively as *current price + a margin tied to consensus strength*, then size at min($\frac{1}{4}$ -Kelly, 5% cap) (Blueprint §8). This is a **proposal**, not an order.

```
[14]: def size_row(r):
    # conservative p estimate: current price nudged up by consensus strength
    edge_margin = min(0.10, 0.04 * (r["score"] / CFG.CONSENSUS_SCORE_CUTOFF))
    p_est = min(0.95, r["cur_price"] + edge_margin)
    eur = position_size_eur(p_est, r["cur_price"], r["score"], CFG.
    ↪CONSENSUS_SCORE_CUTOFF)
    return pd.Series({"p_est": round(p_est, 3), "suggested_eur": eur})

if len(live):
    live = pd.concat([live, live.apply(size_row, axis=1)], axis=1)
    # enforce portfolio cap
    cap = CFG.MAX_TOTAL_DEPLOYED_PCT * CFG.BANKROLL
    live["cum"] = live["suggested_eur"].cumsum()
    live["suggested_eur"] = live.apply(
        lambda r: r["suggested_eur"] if r["cum"] <= cap else 0.0, axis=1)
cols = _
    ↪["title", "outcome", "backers", "dissenters", "score", "median_entry", "cur_price", "p_est", "sugge
live[cols] if len(live) else "No signals today - that is normal. You are a _
    ↪sniper, not a day-trader."
```

```
[14]: 'No signals today - that is normal. You are a sniper, not a day-trader.'
```

1.6 6. Save today's signal sheet

```
[15]: from datetime import datetime
if len(live):
    fname = f"signals_{datetime.now():%Y%m%d}.csv"
    live[cols].to_csv(fname, index=False)
    print("saved", fname)
else:
    print("no signals to save today")
```

no signals to save today

1.6.1 What this notebook deliberately does NOT do

It stops at **proposing** trades. Execution (Blueprint §11) is a separate, riskier build: private-key custody, a daily spend cap, a kill-switch, and a human confirmation step. We wire that up only after the signals are proven by backtest and paper-trading.

Automate the daily run: schedule `03_consensus_signals` to run each morning and email you the CSV — ask and we'll set up the scheduled task + digest.